# SIMATS ENGINEERING
## ASSESSMENT TOOL 1 - HIGH
## (CODING CHALLENGE QUESTIONS)

**COURSE CODE: ITA04**
**COURSE NAME: STATISTICS WITH R PROGRAMMING**

**COURSE OUTCOME COVERED**

**CO2:** *Implement looping constructs, control structures, and recursive functions in R to solve computational problems with efficient program logic. (BL3, BL4)*

Assessment Tool Weightage: 15%

**QUESTIONS: 10** **Total Marks: 100**

ANNEXURE A

**ANALYTICAL PROBLEM SOLVING QUESTIONS**

***Code of Conduct:***

*I Hasya Sai Reddy Venati-192420010 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

1. Write an R program that accepts a numeric vector of daily temperatures and uses an if-else structure to classify each value as “Low” (below 20), “Moderate” (20–30), or “High” (above 30).
2. A college wants to identify students eligible for a scholarship. Write an R program using conditional statements to award eligibility when marks are at least 85 and attendance is at least 80%. Display the result for each student.
3. Write an R program using a for loop to calculate the sum of all even numbers between 1 and n, where n is entered by the user.
4. Develop an R program using a while loop to reverse the digits of a positive integer and display the reversed number.
5. Write an R program using a repeat loop to generate and display random integers until a value greater than 90 is produced. Count and display the number of attempts.
6. Create an R function that accepts a numeric vector and returns a new vector in which positive values are doubled, while zero and negative values remain unchanged. Use an appropriate control structure.
7. Write an R function that accepts a student's mark and returns “Distinction”, “First Class”, “Pass”, or “Fail” using if-else conditions. Define and test suitable mark ranges.
8. Develop an R function with default arguments for basic interest calculation. Let the principal, annual rate, and number of years be inputs, with a suitable default rate and duration. Return the calculated simple interest.
9. Write an R program that uses a for loop and next statement to skip multiples of 5 while displaying numbers from 1 to 50.
10. Write an R program that uses a loop and break statement to search for the first number divisible by both 7 and 11 in a given range.
11. Create an R function that accepts a vector of sales values and returns a named list containing the total sales, average sales, maximum sale, and minimum sale.
12. Write an R function that checks whether a given integer is prime. Use a loop and conditional statements, and test the function with at least three inputs.
13. Develop a recursive R function to calculate the factorial of a non-negative integer. Include a base case and test the function with suitable inputs.
14. Write a recursive R function to calculate the sum of the first n natural numbers. Explain through comments how the base case stops further recursive calls.
15. Create a recursive R function to find the nth Fibonacci number. Validate the input and test the function for several values of n.
16. Write an R program using a while loop to calculate the greatest common divisor (GCD) of two positive integers using the Euclidean algorithm.
17. An electricity board charges different rates for different consumption slabs. Write an R function using if-else statements to calculate a customer's bill based on units consumed and clearly state the slab rates used.
18. Write an R program that processes a vector of exam marks using a loop. Count the number of students who passed (mark at least 40), failed, and scored 75 or above.
19. Create a function that receives a numeric vector and a threshold as arguments, with a default threshold of 50. Return all values greater than or equal to the threshold. Demonstrate both default and user-specified argument calls.

20. Write an R function that accepts a vector and returns NULL if any element is missing; otherwise, return the squared values. Test both cases using is.na() and conditional logic.
21. A delivery service assigns charges based on parcel weight and delivery distance. Write an R program using nested if-else statements to classify the parcel and calculate the delivery charge according to rates you define.
22. Write an R program using a for loop to print a multiplication table for a number entered by the user, from 1 to 12.
23. Create an R function that takes a vector of daily website visits and uses a loop to count how many days had visits above the weekly average. Return the count and the average in a named list.
24. Write an R program that uses a loop to compute the power x^n for a non-negative integer n without using the ** or ^ operator. Include input validation.
25. Design a recursive R function to find the maximum value in a non-empty numeric vector by reducing the vector recursively. State how your function handles a vector with one element.

**RUBRICS FOR CALCULATION**

| Coding Challenge | | | | | |
|---|---|---|---|---|---|
| **Criteria** | **Weightage** | **Level 4 (Exemplary)** | **Level 3 (Proficient)** | **Level 2 (Developing)** | **Level 1 (Beginning)** |
| Problem Understanding | 20% | Clearly interprets problem, identifies all parameters and requirements accurately | Understands problem with minor gaps | Partial understanding; misses key elements | Misinterprets or unable to understand problem |
| Method Selection | 20% | Selects most appropriate method/formula with strong justification | Selects correct method with minor errors | Inappropriate or partially correct method | Unable to select method |
| Solution Process | 25% | Logical, systematic steps with correct approach throughout | Mostly correct steps with minor mistakes | Disorganized steps; multiple errors | No clear process |
| Accuracy of Calculation | 20% | All calculations accurate; correct final answer | Minor calculation errors | Frequent errors; incorrect final answer | No valid calculations |
| Interpretation of Results | 15% | Clearly interprets results with units and engineering relevance | Basic interpretation with minor omissions | Limited or unclear interpretation | No interpretation |

Q1.

```
> temp <- c(15, 25, 35, 20, 30)
>
> for (t in temp) {
+   if (t < 20) {
+     print("Low")
+   } else if (t <= 30) {
+     print("Moderate")
+   } else {
+     print("High")
+   }
+ }
[1] "Low"
[1] "Moderate"
[1] "High"
[1] "Moderate"
[1] "Moderate"
```

Q2.

```
> marks <- c(90, 80, 88, 95)
> attendance <- c(85, 90, 75, 80)
>
> for (i in 1:length(marks)) {
+   if (marks[i] >= 85 && attendance[i] >= 80) {
+     print(paste("Student", i, "Eligible"))
+   } else {
+     print(paste("Student", i, "Not Eligible"))
+   }
+ }
[1] "Student 1 Eligible"
[1] "Student 2 Not Eligible"
[1] "Student 3 Not Eligible"
[1] "Student 4 Eligible"
>
```

Q3.

```
> n <- as.integer(readline("Enter n: "))
Enter n: sum <- 0
Warning message:
NAs introduced by coercion
>
> for (i in 1:n) {
+   if (i %% 2 == 0) {
+     sum <- sum + i
+   }
+ }
Error in 1:n : NA/NaN argument
>
```

Q4.

```
> n <- 10
> sum <- 0
>
> for (i in 1:n) {
+   if (i %% 2 == 0) {
+     sum <- sum + i
+   }
+ }
>
> print(sum)
[1] 30
>
```

Q5.

```
> n <- 1234
> rev <- 0
>
> while (n > 0) {
+   digit <- n %% 10
+   rev <- rev * 10 + digit
+   n <- n %/% 10
+ }
>
> print(rev)
[1] 4321
>
```

Q6.

```
> attempts <- 0
>
> repeat {
+   num <- sample(1:100, 1)
+   attempts <- attempts + 1
+   print(num)
+
+   if (num > 90) {
+     break
+   }
+ }
[1] 12
[1] 76
[1] 61
[1] 60
[1] 30
```

```
[1] 52
[1] 21
[1] 54
[1] 90
[1] 1
[1] 80
[1] 13
[1] 22
[1] 66
[1] 3
[1] 29
[1] 97
>
> print(paste("Attempts:", attempts))
[1] "Attempts: 28"
>
```

Q7.

```
> double_positive <- function(x) {
+   for (i in 1:length(x)) {
+     if (x[i] > 0) {
+       x[i] <- x[i] * 2
+     }
+   }
+   return(x)
+ }
>
> print(double_positive(c(2, -3, 0, 5, -1)))
[1]  4 -3  0 10 -1
>
```

Q8.

```
> grade <- function(mark) {
+   if (mark >= 75) {
+     return("Distinction")
+   } else if (mark >= 60) {
+     return("First Class")
+   } else if (mark >= 40) {
+     return("Pass")
+   } else {
+     return("Fail")
+   }
+ }
>
> print(grade(85))
[1] "Distinction"
> print(grade(85))
[1] "Distinction"
> print(grade(65))
[1] "First Class"
> print(grade(45))
[1] "Pass"
> print(grade(30))
[1] "Fail"
>
```

Q9.

```
> simple_interest <- function(P, R = 5, T = 2) {
+   SI <- (P * R * T) / 100
+   return(SI)
+ }
>
> print(simple_interest(10000))
[1] 1000
> print(simple_interest(10000, 8, 3))
[1] 2400
>
```

Q10.

```
> for (i in 1:50) {
+   if (i %% 5 == 0) {
+     next
+   }
+   print(i)
+ }
[1] 1
[1] 2
[1] 3
[1] 4
[1] 6
[1] 7
[1] 8
[1] 9
[1] 11
[1] 12
```

Q11.

```
> if (!found) {
+   print("No number found")
+ }
> sales_summary <- function(sales) {
+   result <- list(
+     Total = sum(sales),
+     Average = mean(sales),
+     Maximum = max(sales),
+     Minimum = min(sales)
+   )
+   return(result)
+ }
>
> sales <- c(100, 200, 150, 300)
> print(sales_summary(sales))
```

```
$Total
[1] 750

$Average
[1] 187.5

$Maximum
[1] 300

$Minimum
[1] 100
```

Q12.

```
> is_prime <- function(n) {
+   if (n < 2) {
+     return(FALSE)
+   }
+
+   if (n == 2) {
+     return(TRUE)
+   }
+
+   for (i in 2:floor(sqrt(n))) {
+     if (n %% i == 0) {
+       return(FALSE)
+     }
+   }
+
+   return(TRUE)
> print(is_prime(7))
[1] TRUE
> print(is_prime(10))
[1] FALSE
> print(is_prime(13))
[1] TRUE
>
```

Q13.

```
> factorial_r <- function(n) {
+   if (n < 0 || n != floor(n)) {
+     stop("Enter a non-negative integer")
+   }
+
+   if (n == 0 || n == 1) {
+     return(1)
+   }
+
+   return(n * factorial_r(n - 1))
+ }
>
> print(factorial_r(5))
[1] 120
> print(factorial_r(0))
[1] 1
```

Q14.

```
> sum_n <- function(n) {
+   if (n < 0 || n != floor(n)) {
+     stop("Enter a non-negative integer")
+   }
+
+   # Base case stops further recursive calls
+   if (n == 0) {
+     return(0)
+   }
+
+   return(n + sum_n(n - 1))
+ }
>
> print(sum_n(5))
[1] 15
> print(sum_n(10))
[1] 55
```

Q15.

```
> fibonacci <- function(n) {
+   if (n < 0 || n != floor(n)) {
+     stop("Enter a non-negative integer")
+   }
+
+   if (n == 0) {
+     return(0)
+   }
+
+   if (n == 1) {
+     return(1)
+   }
+
+   return(fibonacci(n - 1) + fibonacci(n - 2))
+ }
>
> print(fibonacci(0))
> print(fibonacci(0))
[1] 0
> print(fibonacci(5))
[1] 5
> print(fibonacci(7))
[1] 13
```

Q16.

```
> a <- 24
> b <- 18
>
> while (b != 0) {
+   r <- a %% b
+   a <- b
+   b <- r
+ }
>
> print(paste("GCD =", a))
[1] "GCD = 6"
```

Q17.

```
> bill <- function(units) {
+   if (units < 0) {
+     stop("Units cannot be negative")
+   } else if (units <= 100) {
+     amount <- units * 2
+   } else if (units <= 200) {
+     amount <- 100 * 2 + (units - 100) * 3
+   } else {
+     amount <- 100 * 2 + 100 * 3 +
+       (units - 200) * 5
+   }
+
+   return(amount)
+ }
>
> print(bill(80))
> print(bill(80))
[1] 160
> print(bill(150))
[1] 350
> print(bill(250))
[1] 750
>
```

Q18.

```
> marks <- c(85, 35, 65, 90, 40, 72)
>
> passed <- 0
> failed <- 0
> high <- 0
>
> for (m in marks) {
+   if (m >= 40) {
+     passed <- passed + 1
+   } else {
+     failed <- failed + 1
+   }
+
+   if (m >= 75) {
+     high <- high + 1
+   } else {
+     failed <- failed + 1
+   }
+
+   if (m >= 75) {
+     high <- high + 1
+   }
+ }
>
> print(paste("Passed:", passed))
[1] "Passed: 5"
> print(paste("Failed:", failed))
[1] "Failed: 1"
> print(paste("Scored 75 or above:", high))
[1] "Scored 75 or above: 2"
>
```

Q19.

```
> filter_values <- function(x, threshold = 50) {
+   result <- c()
+
+   for (i in x) {
+     if (i >= threshold) {
+       result <- c(result, i)
+     }
+   }
+
+   return(result)
+ }
>
> x <- c(25, 50, 65, 40, 80)
>
> print(filter_values(x))
```

```
> print(filter_values(x))
[1] 50 65 80
> print(filter_values(x, 60))
[1] 65 80
```

Q20.

```
> square_values <- function(x) {
+   if (any(is.na(x))) {
+     return(NULL)
+   } else {
+     return(x * x)
+   }
+ }
>
> print(square_values(c(2, 3, 4)))
[1]  4  9 16
> print(square_values(c(2, NA, 4)))
NULL
```

Q21.

```
> delivery_charge <- function(weight, distance) {
+   if (weight <= 0 || distance < 0) {
+     stop("Invalid weight or distance")
+   }
+
+   if (weight <= 2) {
+     category <- "Light"
+     charge <- 50
+   } else if (weight <= 5) {
+     category <- "Medium"
+     charge <- 100
+   } else {
+     category <- "Heavy"
+     charge <- 150
+   }
+   if (distance > 20) {
+     charge <- charge + (distance - 20) * 5
+   }
+
+   print(paste("Parcel:", category))
+   return(charge)
+ }
>
> print(delivery_charge(1.5, 10))
[1] "Parcel: Light"
[1] 50
> print(delivery_charge(4, 30))
[1] "Parcel: Medium"
[1] 150
>
```

Q22.

```
> n <- 5
>
> for (i in 1:12) {
+   print(paste(n, "x", i, "=", n * i))
+ }
```

```
[1] "5 x 1 = 5"
[1] "5 x 2 = 10"
[1] "5 x 3 = 15"
[1] "5 x 4 = 20"
[1] "5 x 5 = 25"
[1] "5 x 6 = 30"
[1] "5 x 7 = 35"
[1] "5 x 8 = 40"
[1] "5 x 9 = 45"
[1] "5 x 10 = 50"
[1] "5 x 11 = 55"
[1] "5 x 12 = 60"
>
```

Q23.

```
> website_visits <- function(visits) {
+   avg <- mean(visits)
+   count <- 0
+
+   for (v in visits) {
+     if (v > avg) {
+       count <- count + 1
+     }
+   }
+
+   return(list(
+     Average = avg,
+     Days_Above_Average = count
+   ))
+ }
>
> visits <- c(100, 200, 150, 300, 250)
> print(website_visits(visits))
$Average
[1] 200

$Days_Above_Average
[1] 2
```

Q24.

```
> x <- 2
> n <- 5
> result <- 1
>
> if (n < 0 || n != floor(n)) {
+   stop("Power must be a non-negative integer")
+ }
>
> for (i in 1:n) {
+   result <- result * x
+ }
>
> print(result)
[1] 32
> visits <- c(100, 200, 150, 300, 250)
```

```
> for (i in 1:n) {
+   result <- result * x
+ }
>
> print(result)
[1] 32
> visits <- c(100, 200, 150, 300, 250)
> print(website_visits(visits))
$Average
[1] 200

$Days_Above_Average
[1] 2
```

Q25.

```
find_max <- function(x) {
  if (length(x) == 0) {
    stop("Vector cannot be empty")
  }

  # Base case: one element is the maximum
  if (length(x) == 1) {
    return(x[1])
  }

  max_rest <- find_max(x[-1])

  if (x[1] > max_rest) {
    return(x[1])
  } else {
    return(max_rest)
  }
```

```
+   }
+
+   max_rest <- find_max(x[-1])
+
+   if (x[1] > max_rest) {
+     return(x[1])
+   } else {
+     return(max_rest)
+   }
+ }
>
> print(find_max(c(10, 25, 7, 40, 15)))
[1] 40
> print(find_max(c(8)))
[1] 8
>
```